

NoSQL

Manuela Vázquez

December 2025

1 Introduction

The following report aims to analyze our “Music Streaming Platform” project—which we have modeled, implemented, and improved over the last three submissions—from a different perspective: the database model we will use for its operation.

Until now, our database has been designed following the traditional relational structure; however, we will examine an alternative database model, NoSQL, which offers certain advantages over the one previously used. This will lead us to consider which structures of our project are more suitable to maintain within a relational approach and which ones benefit from being implemented using NoSQL.

In the following sections, we will present a detailed comparison between both models, justifying which parts of our system are more coherent under each approach. This hybrid model will provide our platform with greater efficiency, combining data integrity and consistency with optimized analysis of the large volumes of information generated by each user’s daily activity.

2 Analysis and Justification of NoSQL

2.1 Detailed Comparative Analysis Between the Relational Model and NoSQL for the “Music Streaming Platform”

Let us conduct a detailed study of both models in relation to our project, in order to understand in depth which parts of our system should be migrated to the new database model for the benefit of the overall performance. We will focus on explaining the differences between both models in relation to this music-streaming platform, although it will first be necessary to explain the main differences that exist between the two models:

Let us begin by discussing the relational model, which we have become familiar with in previous submissions. This model organizes data into structured tables, where each entity follows a rigid schema that ensures coherence and consistency. Additionally, the use of primary keys (which uniquely identify each

record) and foreign keys (which establish relationships between tables) reinforces cohesion and referential integrity throughout the system. Broadly speaking, this model is characterized by its fixed and strict structure, its adherence to the ACID properties (Atomicity, Consistency, Isolation, Durability), and its high efficiency when performing join operations. However, it also presents several key limitations, which motivated the need to explore an alternative model capable of addressing these shortcomings.

The NoSQL model breaks away from the logical structure of the relational approach. It emerges as a response to the major challenge of relational databases: their limited effectiveness when handling massive increases in data volume. NoSQL allows the management of millions of records and operations by distributing the workload across multiple nodes, and it supports data that changes frequently. Another fundamental characteristic is horizontal scalability: data and workload can be distributed across several servers, enabling the system to handle large information flows. However, this distributed architecture entails moving away from strict ACID guarantees and adopting eventual consistency instead. Therefore, for components where large amounts of data are inserted or modified continuously, this model is particularly suitable.

As previously mentioned, the parts of our project that should be maintained in the relational model are those that are more structured and stable, where it is essential to ensure data integrity and consistency. Furthermore, this model works very well with relationships between entities, making it indispensable for maintaining accuracy and control over critical data.

On the other hand, structures that receive constant and massive data flows are more suitable for migration to the NoSQL model. Additionally, for structures where the data is dynamic, that is, may change persistently, this model is also appropriate. In the context of our project, all information related to user activity—which continuously changes and grows as users interact with the platform—should be managed from the NoSQL approach.

In conclusion, for our project the most suitable option is to maintain a hybrid model, this hybrid strategy will guide the implementation in MongoDB, allowing the platform to manage large-scale user activity and provide fast and reliable responses, while preserving the accuracy of key data structures. Where key and more rigid structures that require immediate integrity remain in the relational model, while dynamic and large-scale data are managed using NoSQL, which will enhance performance. Next, we will explain in detail which parts are migrated and which are not.